

An Overview of Deep Learning for Curious People

This PDF conversion (v1) was generated on October 10, 2025*

Online version: <https://lilianweng.github.io/posts/2017-06-21-overview/>

Date: **Estimated Reading Time:** min **Author:** Lilian Weng

⁰This file was prepared by Sakura (bili_sakura@zju.edu.cn). Please contact for any issues or copyright concerns. The original source of this file is <https://lilianweng.github.io/posts/2017-06-21-overview/>.

Contents

[Lil'Log](#)

- |
- [Posts](#)
- [Archive](#)
- [Search](#)
- [Tags](#)
- [FAQ](#)

1 An Overview of Deep Learning for Curious People

Date: June 21, 2017 | Estimated Reading Time: 12 min | Author: Lilian Weng

Table of Contents

- Why Does Deep Learning Work Now?
- Deep Learning Models
 - Convolutional Neural Network
 - Recurrent Neural Network
 - RNN: Sequence-to-Sequence Model
 - Autoencoders
- Reinforcement (Deep) Learning
 - Generative Adversarial Network
- Toolkits and Libraries
- How to Learn?
 - Useful resources
 - Blog posts mentioned
 - Interesting blogs worthy of checking
 - Papers mentioned

(The post was originated from my talk for [WiMLDS x Fintech meetup](#) hosted by [Affirm](#).)

I believe many of you have watched or heard of the [games](#) between AlphaGo and professional Go player [Lee Sedol](#) in 2016. Lee has the highest rank of nine dan and many world championships. No doubt, he is one of the best Go players in the world, but he [lost by 1-4](#) in this series versus AlphaGo. Before this, Go was considered to be an intractable game for computers to master, as its simple rules lay out an exponential number of variations in the board positions, many more than what in Chess. This event

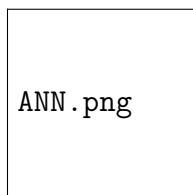


Figure 1: A three-layer artificial neural network. (Image source: <http://cs231n.github.io/convolutional-networks/#conv>)

surely highlighted 2016 as a big year for AI. Because of AlphaGo, much attention has been attracted to the progress of AI.

Meanwhile, many companies are spending resources on pushing the edges of AI applications, that indeed have the potential to change or even revolutionize how we are gonna live. Familiar examples include self-driving cars, chatbots, home assistant devices and many others. One of the secret receipts behind the progress we have had in recent years is deep learning.

2 Why Does Deep Learning Work Now?#

Deep learning models, in simple words, are large and deep artificial neural nets. A neural network (“NN”) can be well presented in a [directed acyclic graph](#): the input layer takes in signal vectors; one or multiple hidden layers process the outputs of the previous layer. The initial concept of a neural network can be traced back to more than [half a century ago](#). But why does it work now? Why do people start talking about them all of a sudden?

The reason is surprisingly simple:

- We have a lot **more data**.
- We have **much powerful computers**.

A large and deep neural network has many more layers + many more nodes in each layer, which results in exponentially many more parameters to tune. Without enough data, we cannot learn parameters efficiently. Without powerful computers, learning would be too slow and insufficient.

Here is an interesting plot presenting the relationship between the data scale and the model performance, proposed by Andrew Ng in his “[Nuts and Bolts of Applying Deep Learning](#)” talk. On a small dataset, traditional algorithms (Regression, Random Forests, SVM, GBM, etc.) or statistical learning does a great job, but once the data scale goes up to the sky, the large NN outperforms others. Partially because compared to a traditional ML model, a neural network model has many more parameters and has the capability to learn complicated nonlinear patterns. Thus we expect the model to pick the most helpful features by itself without too much expert-involved manual feature engineering.

3 Deep Learning Models#

Next, let’s go through a few classical deep learning models.

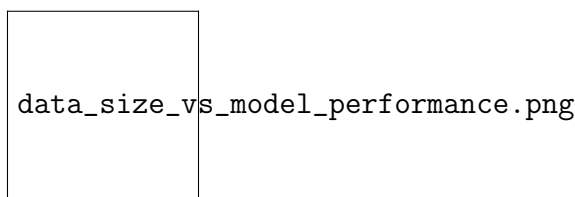


Figure 2: The data scale versus the model performance. (Recreated based on: <https://youtu.be/F1ka6a13S9I>)

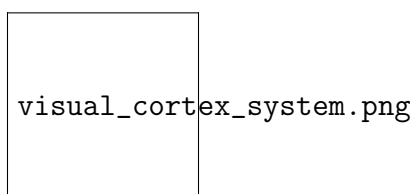


Figure 3: Illustration of the human visual cortex system. (Image source: [Wang & Raj 2017](#))

3.1 Convolutional Neural Network#

Convolutional neural networks, short for “CNN”, is a type of feed-forward artificial neural networks, in which the connectivity pattern between its neurons is inspired by the organization of the visual cortex system. The primary visual cortex (V1) does edge detection out of the raw visual input from the retina. The secondary visual cortex (V2), also called prestriate cortex, receives the edge features from V1 and extracts simple visual properties such as orientation, spatial frequency, and color. The visual area V4 handles more complicated object attributes. All the processed visual features flow into the final logic unit, inferior temporal gyrus (IT), for object recognition. The shortcut between V1 and V4 inspires a special type of CNN with connections between non-adjacent layers: Residual Net ([He, et al. 2016](#)) containing “Residual Block” which supports some input of one layer to be passed to the component two layers later.

Convolution is a mathematical term, here referring to an operation between two matrices. The convolutional layer has a fixed small matrix defined, also called kernel or filter. As the kernel is sliding, or convolving, across the matrix representation of the input image, it is computing the element-wise multiplication of the values in the kernel matrix and the original image values. [Specially designed kernels](#) can process images for common purposes like blurring, sharpening, edge detection and many others, fast and efficiently.

[Convolutional](#) and [pooling](#) (or “sub-sampling” in Fig. 4) layers act like the V1, V2 and V4 visual cortex units, responding to feature extraction. The object recognition reasoning happens in the later fully-connected layers which consume the extracted features.

3.2 Recurrent Neural Network#

A sequence model is usually designed to transform an input sequence into an output sequence that lives in a different domain. Recurrent neural network, short for “RNN”, is suitable for this purpose and has shown tremendous improvement in problems like handwriting recognition, speech recognition, and machine translation ([Sutskever et al. 2011](#), [Liwicki et al. 2007](#)).

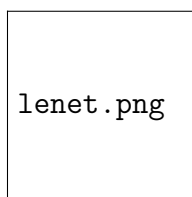


Figure 4: The LeNet architecture consists of two sets of convolutional, activation, and pooling layers, followed by a fully-connected layer, activation, another fully-connected layer, and finally a softmax classifier (Image source: <http://deeplearning.net/tutorial/lenet.html>)

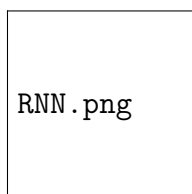


Figure 5: A recurrent neural network with one hidden unit (left) and its unrolling version in time (right). The unrolling version illustrates what happens in time: s_{t-1} , s_t , and s_{t+1} are the same unit with different states at different time steps $t-1$, t , and $t+1$. (Image source: [LeCun, Bengio, and Hinton, 2015](#); Fig. 5)

A recurrent neural network model is born with the capability to process long sequential data and to tackle tasks with context spreading in time. The model processes one element in the sequence at one time step. After computation, the newly updated unit state is passed down to the next time step to facilitate the computation of the next element. Imagine the case when an RNN model reads all the Wikipedia articles, character by character, and then it can predict the following words given the context.

However, simple perceptron neurons that linearly combine the current input element and the last unit state may easily lose the long-term dependencies. For example, we start a sentence with “Alice is working at ...” and later after a whole paragraph, we want to start the next sentence with “She” or “He” correctly. If the model forgets the character’s name “Alice”, we can never know. To resolve the issue, researchers created a special neuron with a much more complicated internal structure for memorizing long-term context, named “[Long-short term memory \(LSTM\)](#)” cell. It is smart enough to learn for how long it should memorize the old information, when to forget, when to make use of the new data, and how to combine the old memory with new input. This [introduction](#) is so well written that I recommend everyone with interest in LSTM to read it. It has been officially promoted in the [Tensorflow documentation](#) ;-)

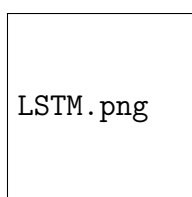


Figure 6: The structure of a LSTM cell. (Image source: <http://colah.github.io/posts/2015-08-Understanding-LSTMs>)

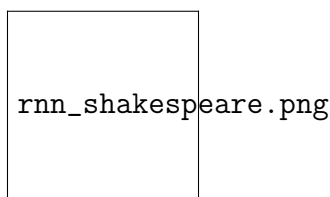


Figure 7: A character-based recurrent neural network model writes like a Shakespeare. (Image source: <http://karpathy.github.io/2015/05/21/rnn-effectiveness>)

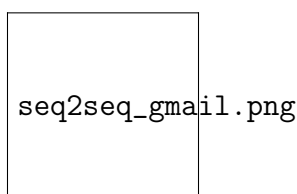


Figure 8: A sequence-to-sequence model for generating Gmail auto replies. (Image source: <https://research.googleblog.com/2015/11/computer-respond-to-this-email.html>)

To demonstrate the power of RNNs, [Andrej Karpathy](#) built a character-based language model using RNN with LSTM cells. Without knowing any English vocabulary beforehand, the model could learn the relationship between characters to form words and then the relationship between words to form sentences. It could achieve a decent performance even without a huge set of training data.

3.3 RNN: Sequence-to-Sequence Model#

The [sequence-to-sequence model](#) is an extended version of RNN, but its application field is distinguishable enough that I would like to list it in a separated section. Same as RNN, a sequence-to-sequence model operates on sequential data, but particularly it is commonly used to develop chatbots or personal assistants, both generating meaningful response for input questions. A sequence-to-sequence model consists of two RNNs, encoder and decoder. The encoder learns the contextual information from the input words and then hands over the knowledge to the decoder side through a “**context vector**” (or “thought vector”, as shown in Fig 8.). Finally, the decoder consumes the context vector and generates proper responses.

3.4 Autoencoders#

Different from the previous models, autoencoders are for unsupervised learning. It is designed to learn a **low-dimensional** representation of a **high-dimensional** data set, similar to what [Principal Components Analysis \(PCA\)](#) does. The autoencoder model tries to learn an approximation function $f(x) \approx x$ to reproduce the input data. However, it is restricted by a bottleneck layer in the middle with a very small number of nodes. With limited capacity, the model is forced to form a very efficient encoding of the data, that is essentially the low-dimensional code we learned.

[Hinton and Salakhutdinov](#) used autoencoders to compress documents on a variety of topics. As shown in Fig 10, when both PCA and autoencoder were applied to reduce

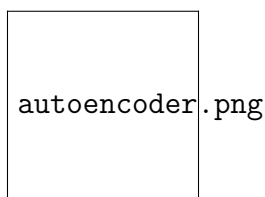


Figure 9: An autoencoder model has a bottleneck layer with only a few neurons. (Image source: Geoffrey Hinton’s Coursera class “[Neural Networks for Machine Learning](#)” - [Week 15](#))

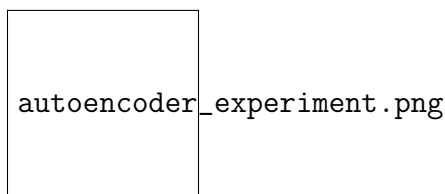


Figure 10: The outputs of PCA (left) and autoencoder (right) when both try to compress documents into two numbers. (Image source: [Hinton & Salakhutdinov 2006](#))

the documents onto two dimensions, autoencoder demonstrated a much better outcome. With the help of autoencoder, we can do efficient data compression to speed up the information retrieval including both documents and images.

4 Reinforcement (Deep) Learning#

Since I started my post with AlphaGo, let us dig a bit more on why AlphaGo worked out. [Reinforcement learning](#) (“RL”) is one of the secrets behind its success. RL is a subfield of machine learning which allows machines and software agents to automatically determine the optimal behavior within a given context, with a goal to maximize the long-term performance measured by a given metric.

7 Citation

Please cite this work as:

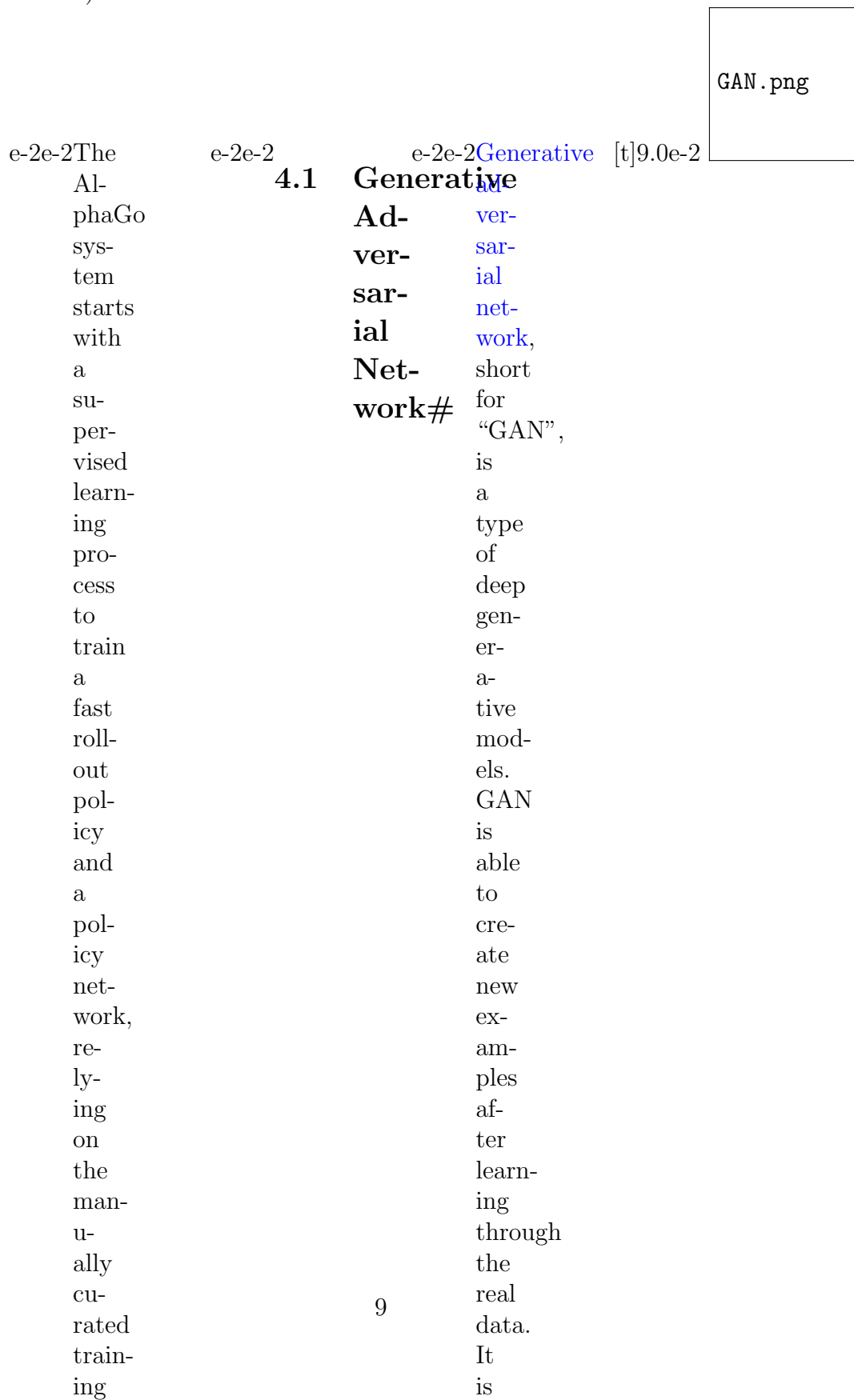
Weng, Lilian. “[Title](#)”. Lil’Log (May 2025). <https://lilianweng.github.io/posts/2017-06-21-overview/>

Or use the BibTeX citation:

```
@article{weng2025,  
  title = {},  
  author = {Weng, Lilian},  
  journal = {lilianweng.github.io},  
  year = {2025}, month = {May},  
  url = {}  
}
```




Figure 11: AlphaGo neural network training pipeline and architecture. (Image source: [Silver et al. 2016](#))



8 References

Tags: [tag1](#) [tag2](#)

[⟨ Title Title ⟩](#)

Share this article: [Twitter](#) | [LinkedIn](#) | [Reddit](#) | [Facebook](#) | [WhatsApp](#) | [Telegram](#)